

Backend Assessment: The Asset Search Index

1. Context

Our clients keep tens of thousands of assets in their library — documents, images, videos — and today the only way to find one again is to remember what it was called. Names are rarely helpful:

`Q3_deck_FINAL_v2` tells nobody anything.

The Proposal

We're building a Proof of Concept (PoC) for a **semantic index**: a service that keeps a searchable picture of the library as it changes, so people can find an asset by describing it rather than by remembering its name.

The Goal

Build a **semantic index** over the library, and an endpoint that searches it in natural language. That's the whole ask.

How you get there is the interesting part, and it's what we'll dig into: where the data comes from, how changes to it reach your service, how you keep the index honest once they do. Those calls are yours — we care about how you reason about them more than about how much you build, so tell us what you chose and why.

The data

There is no fixture — **you build the sample data yourself**. An asset is just three fields:

```
JSON
{
  "id": "ast_8831",
  "name": "Q3_deck_FINAL_v2",
  "description": "A quarterly business review deck: revenue against target by region, a churn breakdown for enterprise accounts, and a closing slide of hiring plans for the next two quarters."
}
```

- `description` is **already written** for you by an upstream service. No language model needed — it is simply the text you index.
- `name` is whatever a human typed when they saved the asset: occasionally meaningful, usually not. What you do with it is your call.

A hundred or so assets across a handful of subjects is plenty — enough that a semantic search has something interesting to choose between.

2. What to build

1. Index

- Embed each **description** and store the vector with whatever else a search result needs.
- An asset is searchable by its **current** description, not a stale one. A deleted asset is not searchable at all.
- Search reads what you indexed, not your source data re-read off disk.

2. Search

- **GET /search?q=<natural language>**, returning the top ten with their scores.
 - Rank by semantic similarity, so a search for **?q=hiring** can surface an asset whose description mentions headcount plans without containing that word.
-

Acceptance Criteria (AC)

- [] **Lifecycle:** created assets are searchable, updated assets are searchable by their current description, deleted assets are not searchable at all.
- [] **Smart search:** a query whose words appear in none of your descriptions still returns the right assets. Put the request and its response in the README.
- [] **Runnable:** one documented command takes your sample data all the way through to a searchable index.

Notes

- **Please don't spend more than four hours on this.** We're not looking for a finished product, and we'd rather you stopped early than polished.
- The first criteria matter most. If you run out of time, cut scope — then say so in the README and tell us what you'd do next.
- **Tests matter** — the happy path, plus whichever edge cases you judge worth covering.

- **Out of scope:** authentication, a frontend, deployment, CI, and anything beyond the top-ten search response. Don't spend your four hours there.
 - Feel free to add your own twist beyond the criteria.
-

Technical Information

Stack

- **Language and framework:** whatever you'll do your best work in.
- **Storage and vectors:** your call. At a couple of hundred records a full scan over stored vectors is perfectly fine — we're more interested in what you'd reach for at ten million, which we can discuss in the demo.
- **Embeddings:** a model running on your own machine — no keys to exchange, no per-call cost. Read where it lives from the environment:

None

EMBEDDING_BASE_URL=http://host.docker.internal:11434/v1

EMBEDDING_MODEL=nomic-embed-text

EMBEDDING_DIMENSIONS=768

- Anything serving an OpenAI-compatible `/v1/embeddings` will do — Ollama, LM Studio, whatever you already have. Name the model in your README and we'll run the same one, so nothing needs to be model-agnostic. If your app runs in a container, remember that `localhost` is the container: use `host.docker.internal`.

Ground Rules

- Feel free to use AI tools (Cursor, Claude, Copilot) to move faster. Then show us **how you direct them and how you verify their output** — that's a skill we're assessing, not something to hide.
- A README is part of the deliverable: what you built, the calls you made and why, what you deliberately left out, and what you'd change with another week.

Delivery

Send us a link to a **GitHub repository** we can clone and run locally.

- The README must name the embedding model to run, the setup commands, how to run the tests, how to load the data, and one example search request with its response.
 - Getting it running should take one pass through your README — no code changes, no undocumented steps.
 - Everything needed lives in the repo, sample data committed.
 - We'd rather see the **real commit history** than one squashed "initial commit".
 - If something is broken or unfinished when you stop, say so in the README. We read that as accuracy, not as a missing feature.
-

The Demo

After you submit, be ready to walk us through your choices. Please be prepared to discuss:

1. **Architecture** — the shape you chose, how the changes get in, and what you'd do differently if this were a live stream from another team's service.
2. **Correctness** — what can go wrong between a change happening upstream and the asset being searchable, and what your design does about each case.
3. **Your test data** — how you built it, and which awkward cases you deliberately included.
4. **Indexing choices** — what you embed, what you store next to the vector, and what you'd do with the asset's **name** if you wanted better results.
5. **Ranking** — how you'd tell whether a change made search better or worse.
6. **Scale** — what breaks first at a million assets, what the embedding work costs in time and compute, and what you'd do about it.

If you have any questions or need clarification, feel free to reach out to us.

You got this!